React desde cero

Ejercicios propuestos: enunciados y pistas

Cada ejercicio se puede resolver en JavaScript (`App.jsx`) o en TypeScript (`App.tsx`). El código inicial de cada uno está en la página interactiva de la clase.

Intentá resolverlo primero sin mirar la pista.

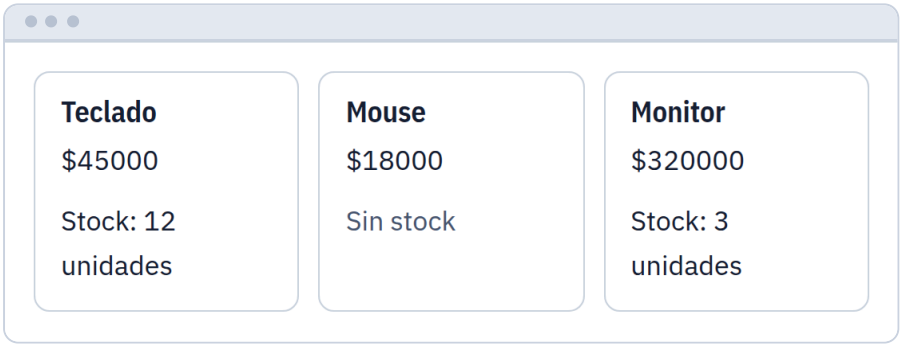
1 Tarjetas de producto

Props

Creá un componente `Producto` que reciba `nombre`, `precio` y `stock`, y mostralo tres veces con datos distintos. Si el stock es 0, en lugar de la cantidad tiene que decir "Sin stock".

> **Pista**
>
> Desestructurá las props en la firma: `function Producto({ nombre, precio, stock })`.
> Para el stock usá un ternario: `stock === 0 ? … : …`.

Así debería quedar



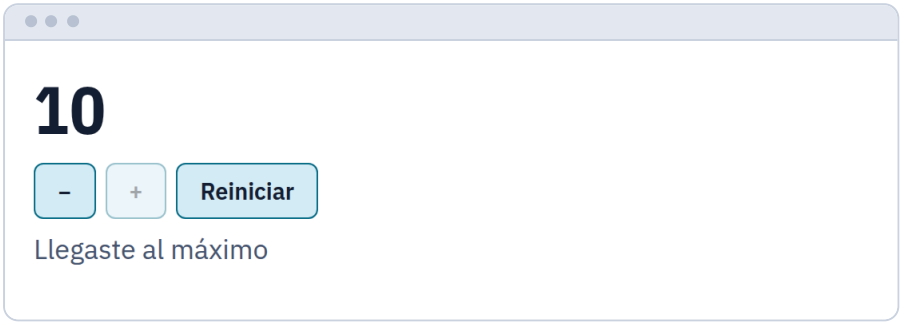
Las tres tarjetas, una de ellas sin stock.

2 Contador con límites useState

Un contador que va de 0 a 10, con botones para restar, sumar y reiniciar. Nunca puede salirse del rango. Extra: deshabilitá cada botón cuando se llega al límite.

> **Pista**
> Los botones aceptan la prop `disabled`: `<button disabled={valor === 10}>`. Y antes de sumar podés chequear `if (valor < 10)`.

Así debería quedar



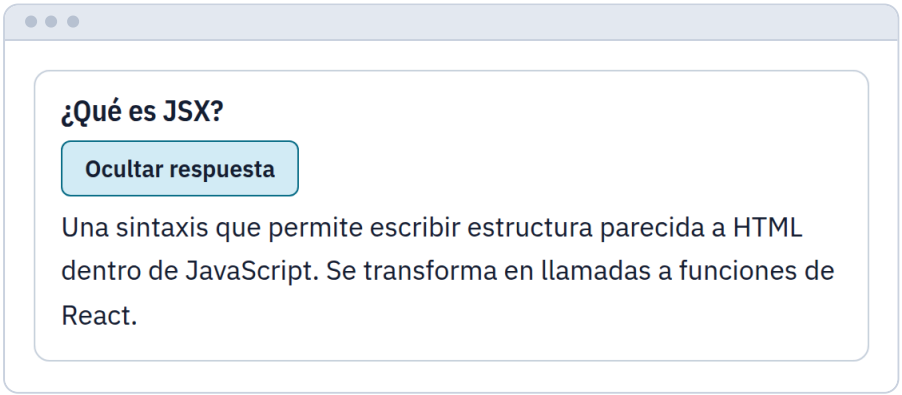
Al llegar a 10: el botón + queda deshabilitado.

3 Mostrar y ocultar Condicionales

Armá una tarjeta de pregunta y respuesta. La respuesta arranca oculta y un botón la muestra u oculta. El texto del botón cambia entre "Mostrar respuesta" y "Ocultar respuesta".

> **Pista**
> Un estado booleano alcanza: `const [visible, setVisible] = useState(false)`. Para alternar: `setVisible(!visible)`.

Así debería quedar



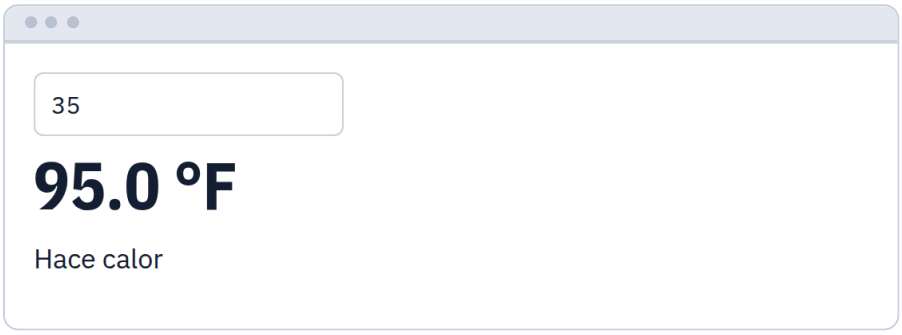
Con la respuesta visible.

4 Conversor de temperatura Inputs controlados

Un input donde se escriben grados Celsius y, debajo, el equivalente en Fahrenheit que se actualiza mientras se escribe. Fórmula: F = C × 9/5 + 32. Extra: si la temperatura supera los 30 °C, mostrá "Hace calor".

> **Pista**
>
> El valor del input es texto. Convertilo con `Number(celsius)` antes de calcular. Fahrenheit no necesita estado propio: se calcula a partir de Celsius en cada renderizado.

Así debería quedar



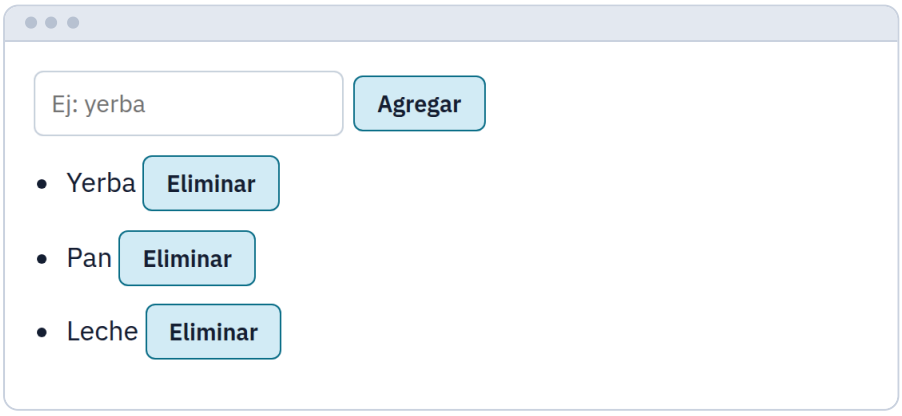
Con 35 °C ingresados.

5 Lista de compras Listas y estado

Un input y un botón "Agregar" que suma el texto a una lista. Cada ítem tiene un botón para eliminarlo. No se pueden agregar ítems vacíos y, si la lista está vacía, se muestra "No hay nada para comprar".

> **Pista**
>
> Nunca modifiques el array del estado con `push`. Creá uno nuevo: para agregar, `[...items, nuevo]`; para eliminar, `items.filter(i => i.id !== id)`. Como id podés usar `Date.now()`.

Así debería quedar



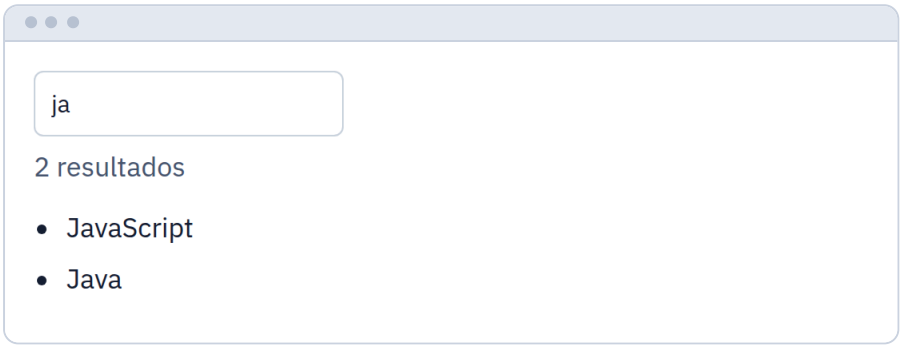
Después de agregar tres ítems.

6 Buscador Filtrado

Dada una lista de lenguajes de programación, mostrá un input de búsqueda que filtre la lista mientras se escribe, sin distinguir mayúsculas de minúsculas. Mostrá también cuántos resultados hay.

> **Pista**
>
> Pasá ambos textos a minúsculas y usá `includes`: `l.toLowerCase().includes(busqueda.toLowerCase())`.

Así debería quedar



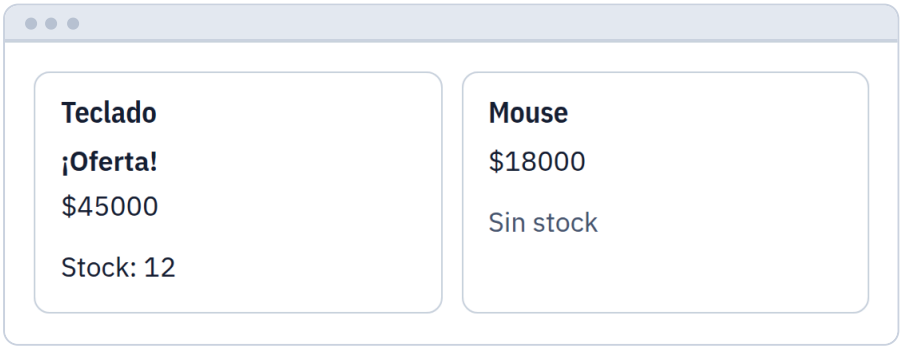
Buscando «ja».

7 Tipar con TypeScript TypeScript

Pasá el componente `Producto` del ejercicio 1 a TypeScript: definí una `interface ProductoProps` y usala en la firma. Agregá una prop opcional `oferta` de tipo booleano que, si es verdadera, muestre "¡Oferta!".

> **Pista**
>
> Una prop opcional se declara con signo de pregunta: `oferta?: boolean;`. Recordá que en la página de la clase los tipos no se verifican; copiá tu solución a un proyecto Vite para ver los errores reales en VS Code.

Así debería quedar



Con la prop oferta activada en el teclado.